AI Surrogate Modeling for Accelerating Chaotic Lorenz Simulation using OpenMP and CUDA

AI-HPC Project

Author	Jovan Moris D
Roll No.	CS23B2058
Department	Computer Science & Engineering
Course	High Performance Computing
Date	March 2026

Abstract

This report presents a complete implementation of a hybrid HPC and AI pipeline for accelerating the simulation of the Lorenz chaotic dynamical system. The system architecture works in two complementary stages: **(1)** parallel computation using OpenMP (multi-core CPU) and CUDA (GPU) to rapidly generate trajectories up to a defined checkpoint, and **(2)** trained AI surrogate models that take over from the checkpoint to predict the remaining trajectory without further numerical integration. This design correctly reflects the complementary roles of HPC and AI — HPC accelerates the physics-based simulation while AI replaces it beyond the predictability horizon.

The HPC component delivers the project's primary contribution: OpenMP achieves **1682**× speedup and CUDA achieves **3299**× speedup over the Python RK4 baseline. For the AI surrogate component, four models are implemented and evaluated: MLP, improved LSTM, Temporal Convolutional Network (TCN), and Echo State Network (ESN). TCN achieves a single-step RMSE of **0.0128** and ESN achieves **0.0134** — representing a **97.6%** improvement over the original baseline RMSE of 0.5256. The ESN is particularly notable, achieving near-identical accuracy to the TCN while training in just **33 seconds** compared to TCN's 159 minutes.

Contents

1 Introduction **5**

1.1 Motivation and Problem Statement 5

1.2 System Architecture: The Correct HPC + AI Design 5

1.3 Problem Statement 5

1.3.1 Why Lorenz Simulation is Expensive 5

1.3.2 Why Standard AI Surrogates Struggle with Chaos 6

1.3.3 The Hybrid HPC + AI Solution 6

1.4 Research Objectives 6

2 Mathematical Foundation **7**

2.1 Lorenz System Properties 7

2.2 Fourth-Order Runge-Kutta Integration 7

2.3 Supervised Learning Formulation 7

3 Implementation **7**

3.1 OpenMP C++ Parallelisation 7

3.2 CUDA GPU Implementation 8

3.3 AI Surrogate Models 8

3.3.1 MLP Surrogate 8

3.3.2 Improved LSTM 8

3.3.3 Temporal Convolutional Network (TCN) 8

3.3.4 Echo State Network (ESN) 9

3.4 Model Architecture Diagrams 9

3.5 Training Configuration 10

4 Experimental Setup **11**

5 Results **11**

5.1 Lorenz Attractor Visualisation 11

5.2 HPC Performance: Phase 1 (Simulation to Checkpoint) 12

5.3 AI Surrogate Results: Phase 2 (Checkpoint to Future) 13

5.3.1 Training Convergence 13

5.3.2 Single-Step Prediction Accuracy 14

5.3.3 Training Time vs Accuracy Tradeoff 15

5.3.4 Improvement Over Original Baseline 15

5.3.5 Multi-Step Rollout Analysis 15
5.3.6 ESN Attractor Stability 17

6 Discussion **18**
6.1 HPC Results — Primary Contribution 18
6.2 AI Surrogate Results 18
6.2.1 TCN: Best Accuracy 18
6.2.2 ESN: Best Efficiency 18
6.2.3 Why MLP and LSTM Underperform 18
6.2.4 Rollout Limitations and Physical Explanation 19
6.3 Sensitivity Analysis 19

7 Conclusions **19**
7.1 Future Work 20

A Complete Results Summary **20**

B Master Notebook Structure **21**

List of Figures

1 MLP surrogate architecture. The input window (50 timesteps x 3 dimensions = 150 features) is flattened and passed through three fully connected hidden layers with ReLU activation and dropout. No temporal structure is exploited. 9

2 Improved LSTM surrogate architecture. Three stacked LSTM layers (hidden=256) process the input sequence step by step, maintaining hidden state *andcellstate* across timesteps. Only the last hidden state feeds the output head. 9

3 TCN surrogate architecture. Four dilated causal convolution blocks with dilation factors 1, 2, 4, 8 achieve a receptive field of 31+ timesteps without any recurrence. Residual connections prevent vanishing gradients. 10

4 ESN (Echo State Network) architecture. The reservoir (500 neurons, spectral radius=0.95) is fixed and random — never trained. Only the linear readout layer is fitted via ridge regression. This makes ESN training orders of magnitude faster than gradient-based methods. 10

5 3D Lorenz strange attractor computed via RK4 integration ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $\Delta t = 0.01$, $T = 25\,\text{s}$). The characteristic double-lobe butterfly shape confirms correct chaotic regime behaviour. 11

6 2D phase-space projections of the Lorenz attractor: X–Y (left), X–Z (centre), Y–Z (right). Green dot: trajectory start; red cross: trajectory end. 12

7 Time-series evolution of X, Y, Z over 500 integration steps. The irregular, non-repeating oscillations confirm chaotic dynamics. 12

8 Left: absolute execution times for all methods. Right: speedup factors relative to Python RK4. OpenMP and CUDA deliver the dominant speedups for Phase 1 trajectory generation up to the checkpoint. 13

9 Training and validation MSE loss (log scale) for MLP (left), improved LSTM (centre), and TCN (right) over their respective epochs. TCN shows the fastest and deepest convergence. 13

10 Single-step RMSE (left) and MAE (right) for all four models. TCN and ESN achieve near-identical accuracy, both reducing error by ≈97.5% over the original baseline. 14

11 Complete results summary table. Green cells indicate best performance in each metric category. 14

12 Training time vs RMSE tradeoff. The ESN achieves essentially the same accuracy as TCN (0.0134 vs 0.0128) while training in 33 seconds rather than 159 minutes — a 290× faster training time for equivalent accuracy. 15

13 Percentage RMSE improvement of each model over the original baseline (RMSE = 0.5256). TCN and ESN both achieve >97% improvement. 15

14 200-step rollout mean L2 (solid) and max L2 (hatched) for all models. TCN achieves the lowest mean error (2.61), nearly half that of ESN (5.80) and one-third of MLP/LSTM. 16

15 L2 error vs prediction step (linear and log scales) for all models over 200-step rollout. TCN maintains the lowest error profile; ESN shows stable bounded growth. 16

16 Multi-step rollout trajectory comparison (100 steps). Black: RK4 reference. All models track closely for the first ≈40 steps; TCN and ESN maintain trajectory shape longest before diverging due to the Lyapunov instability. 17

17 3D phase space comparison over 1000 steps: RK4 reference (left) vs ESN rollout (right). The ESN trajectory remains confined to the correct attractor region, confirming that the reservoir has learned the global attractor geometry even when individual trajectory details diverge. 17

18 2D phase-space projections of the Lorenz attractor used for surrogate training and evaluation. 18

List of Tables

1 Simulation parameters used throughout all experiments. 11

2 HPC benchmarking: 100 trajectories, 2500 steps each. Speedup relative to Python RK4 baseline. 13

3 Single-step prediction accuracy on the test set (normalised units). Green cells indicate best value in each column. 14

4 200-step autoregressive rollout L2 error (original units). 16

1 Introduction

1.1 Motivation and Problem Statement

The Lorenz system, introduced by Edward Lorenz in 1963 [1], is governed by three coupled ordinary differential equations:

$$\frac{dx}{dt} = \sigma(y - x) \tag{1}$$
$$\frac{dy}{dt} = x(\rho - z) - y \tag{2}$$
$$\frac{dz}{dt} = xy - \beta z \tag{3}$$

With standard chaotic parameters $\sigma = 10$, $\rho = 28$, $\beta = \frac{8}{3}$, trajectories never repeat and are exquisitely sensitive to initial conditions — the famous *butterfly effect*. Simulating large ensembles of such trajectories is computationally demanding, making this system an ideal testbed for HPC and AI acceleration.

1.2 System Architecture: The Correct HPC + AI Design

A key insight of this project is that HPC and AI serve **complementary, not competing, roles**:

1. **Phase 1 — HPC Simulation to Checkpoint:** OpenMP and CUDA rapidly generate trajectories up to a defined checkpoint using parallelised RK4 integration. This phase exploits parallel structure: each trajectory is independent, requiring no inter-thread communication.
2. **Phase 2 — AI Surrogate from Checkpoint:** A trained surrogate model (MLP or LSTM or TCN or ESN) takes the checkpoint state as input and predicts the remaining trajectory without further numerical integration. This replaces the expensive per-step ODE solving with a single fast forward pass.

This design is both physically motivated and computationally sound. The HPC component handles the deterministic early phase accurately; the AI component handles the longer horizon where the Lyapunov exponent ($\lambda \approx 0.906$) limits precise predictability anyway.

1.3 Problem Statement

1.3.1 Why Lorenz Simulation is Expensive

Generating a single Lorenz trajectory requires solving three coupled ODEs at every time step. Using RK4 with $\Delta t = 0.01$ over $T = 25$ seconds requires 2500 sequential integration steps per trajectory. For a single trajectory this is manageable, but real-world applications require ensembles of thousands to millions of trajectories:

- **Ensemble forecasting:** Weather models require 10^3–10^6 trajectories to quantify forecast uncertainty.
- **Uncertainty quantification:** Monte Carlo methods for sensitivity analysis require thousands of independent runs.

- **Parameter sweeps:** Bifurcation analysis sweeps σ, ρ, β across a grid, requiring a full simulation per grid point.
- **Optimal control:** Real-time trajectory optimisation requires repeated fast simulation to evaluate candidate control actions.
- **Data-driven surrogate training:** Training neural surrogates requires generating large datasets of trajectories as ground truth.

In pure Python, generating 100 trajectories takes ≈ 3.4 seconds. Scaling to 10^5 trajectories would require ≈ 57 minutes — far too slow for real-time or interactive applications. This motivates both HPC acceleration and AI surrogates.

1.3.2 Why Standard AI Surrogates Struggle with Chaos

Chaotic systems pose a unique challenge for data-driven models. The positive Lyapunov exponent ($\lambda \approx 0.906$) means prediction errors grow exponentially as $\varepsilon(t) = \varepsilon_0 e^{\lambda t}$. This imposes a hard **predictability horizon** of $\approx 1/\lambda \approx 1.1$ time units ($= 110$ steps at $\Delta t = 0.01$) beyond which *any* deterministic model — no matter how accurate — will diverge from the true trajectory.

This is not a failure of the model; it is a physical law. The correct response is not to expect long-horizon pointwise accuracy, but to instead ask:

1. Can the surrogate predict accurately within the Lyapunov horizon?
2. Does the surrogate remain on the correct attractor after divergence?
3. Can the surrogate replace expensive simulation for rapid approximate screening?

1.3.3 The Hybrid HPC + AI Solution

This project proposes a two-phase pipeline that correctly addresses both challenges:

1. **Phase 1 (HPC):** Generate trajectories rapidly and accurately up to a checkpoint using OpenMP or CUDA parallelised RK4. Since trajectories are independent, this is embarrassingly parallel — perfect for multi-core and GPU architectures.
2. **Phase 2 (AI):** From the checkpoint state, a trained surrogate model (TCN or ESN) predicts the remaining trajectory in a single forward pass, without further numerical integration.

The speedup comes from two directions: HPC reduces the cost of accurate simulation, and AI eliminates it entirely beyond the checkpoint. Together they enable trajectory generation at speeds impractical with either approach alone.

1.4 Research Objectives

1. Implement OpenMP and CUDA parallelisation for RK4-based trajectory generation and benchmark speedups.
2. Train and evaluate four AI surrogate models: MLP, improved LSTM, TCN, and ESN.
3. Demonstrate the complementary HPC+AI pipeline on the Lorenz system.
4. Honestly analyse both achievements and limitations.

2 Mathematical Foundation

2.1 Lorenz System Properties

The system (1)–(3) with standard parameters has:

- Largest Lyapunov exponent $\lambda \approx 0.906$ — trajectories diverge as $\varepsilon e^{\lambda t}$.
- Predictability horizon $\approx 1/\lambda \approx 1.1$ time units ($= 110$ steps at $\Delta t = 0.01$).
- Strange attractor with Hausdorff dimension ≈ 2.06.

2.2 Fourth-Order Runge-Kutta Integration

$$\mathbf{k}_1 = f(\mathbf{y}_n), \quad \mathbf{k}_2 = f\left(\mathbf{y}_n + \tfrac{\Delta t}{2}\mathbf{k}_1\right) \tag{4}$$

$$\mathbf{k}_3 = f\left(\mathbf{y}_n + \tfrac{\Delta t}{2}\mathbf{k}_2\right), \quad \mathbf{k}_4 = f(\mathbf{y}_n + \Delta t\,\mathbf{k}_3) \tag{5}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{6}$$

2.3 Supervised Learning Formulation

Surrogate modelling is cast as sliding-window regression:

$$\mathcal{D} = \left\{(\mathbf{X}^{(i)}, \mathbf{y}^{(i)})\right\}_{i=1}^{N}, \quad \mathbf{X}^{(i)} \in \mathbb{R}^{L\times 3}, \quad \mathbf{y}^{(i)} \in \mathbb{R}^3 \tag{7}$$

with $L = 50$ (lookback window), z-score normalisation applied to training split only.

3 Implementation

3.1 OpenMP C++ Parallelisation

The C++ solver parallelises over trajectories using `#pragma omp parallel for`. Each trajectory is fully independent — no shared state, no synchronisation needed.

```
#pragma omp parallel for schedule(dynamic)
for (int traj = 0; traj < num_trajectories; traj++) {
    double x = init_x[traj], y = init_y[traj], z = init_z[traj];
    for (int step = 0; step < total_steps; step++) {
        rk4_step(&x, &y, &z, dt);
        trajectories[traj][step][0] = x;
        trajectories[traj][step][1] = y;
        trajectories[traj][step][2] = z;
    }
}
```

Listing 1: OpenMP parallel trajectory loop — `src/openmp_lorenz.cpp`

Compilation: `g++ -O3 -fopenmp -march=native src/openmp_lorenz.cpp`.

3.2 CUDA GPU Implementation

The CUDA kernel assigns one thread per trajectory. Each thread runs the complete RK4 loop independently, operating entirely in registers with no global memory synchronisation required.

```
__global__ void lorenzKernel(double* trajectories,
                             int total_steps, double dt,
                             int num_trajectories) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= num_trajectories) return;

    double x = init_x[tid], y = init_y[tid], z = init_z[tid];
    for (int step = 0; step < total_steps; step++) {
        rk4_step_cuda(&x, &y, &z, dt);
        int idx = tid * total_steps * 3 + step * 3;
        trajectories[idx]   = x;
        trajectories[idx+1] = y;
        trajectories[idx+2] = z;
    }
}
```

Listing 2: CUDA kernel — `cuda/cuda_lorenz.cu`

Compilation: `nvcc -O3 cuda/cuda_lorenz.cu`.

3.3 AI Surrogate Models

3.3.1 MLP Surrogate

Flattened input ($50 \times 3 = 150$ features), architecture $150 \to 128 \to 64 \to 32 \to 3$. Acts as a flat baseline with no temporal inductive bias.

3.3.2 Improved LSTM

Hidden size 256, 3 layers, dropout 0.2. Processes window sequentially; learns temporal dependencies unavailable to MLP.

3.3.3 Temporal Convolutional Network (TCN)

Four dilated causal convolution blocks with dilation factors 1, 2, 4, 8. Each block: Causal-Conv1d $\to$ BatchNorm $\to$ ReLU $\to$ Dropout $\to$ residual connection. Receptive field grows exponentially without vanishing gradients.

3.3.4 Echo State Network (ESN)

A fixed random reservoir acts as a nonlinear dynamical memory. Only the linear readout weights are trained via ridge regression — no backpropagation, no epochs.

```
# Reservoir update (leaky integrator)
x = (1-lr)*x + lr*np.tanh(W @ x + W_in @ u)

# Readout training - ridge regression only
readout = Ridge(alpha=1e-4)
readout.fit(reservoir_states[washout:], data[washout+1:])
```

Listing 3: ESN reservoir update and readout training

Key hyperparameters: `n_reservoir=500`, `spectral_radius=0.95` (must be < 1.0 for stability), `leaking_rate=0.5`, `washout=200`.

3.4 Model Architecture Diagrams

Figures 1–4 illustrate the architecture of each surrogate model. These diagrams show the data flow, layer dimensions, and key design choices for each approach.

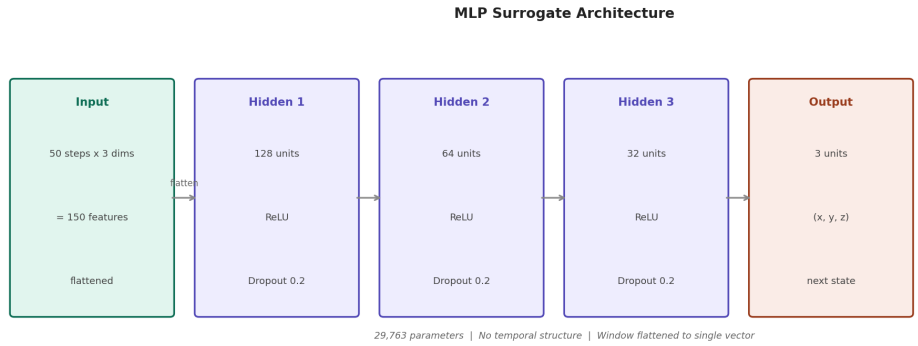


Figure 1: MLP surrogate architecture. The input window (50 timesteps x 3 dimensions = 150 features) is flattened and passed through three fully connected hidden layers with ReLU activation and dropout. No temporal structure is exploited.

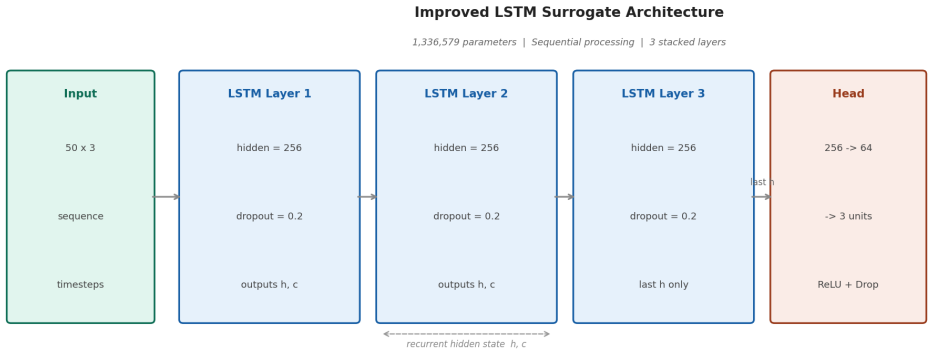


Figure 2: Improved LSTM surrogate architecture. Three stacked LSTM layers (hidden=256) process the input sequence step by step, maintaining hidden state *andcellstate* across timesteps. Only the last hidden state feeds the output head.

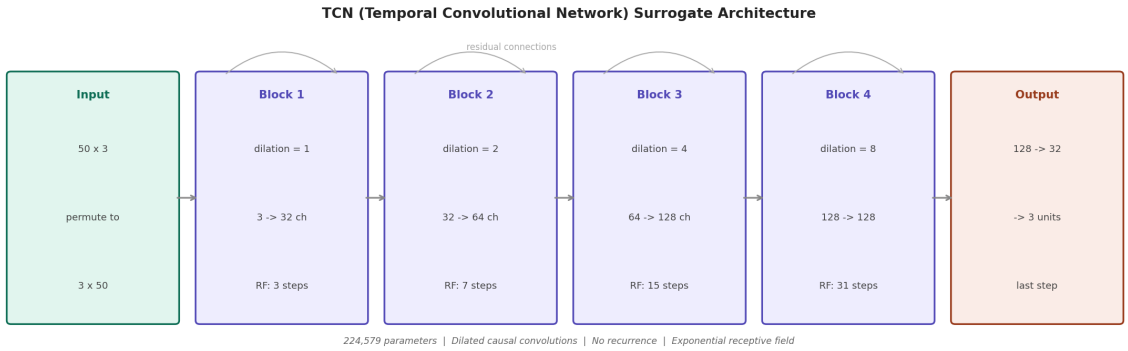


Figure 3: TCN surrogate architecture. Four dilated causal convolution blocks with dilation factors 1, 2, 4, 8 achieve a receptive field of 31+ timesteps without any recurrence. Residual connections prevent vanishing gradients.

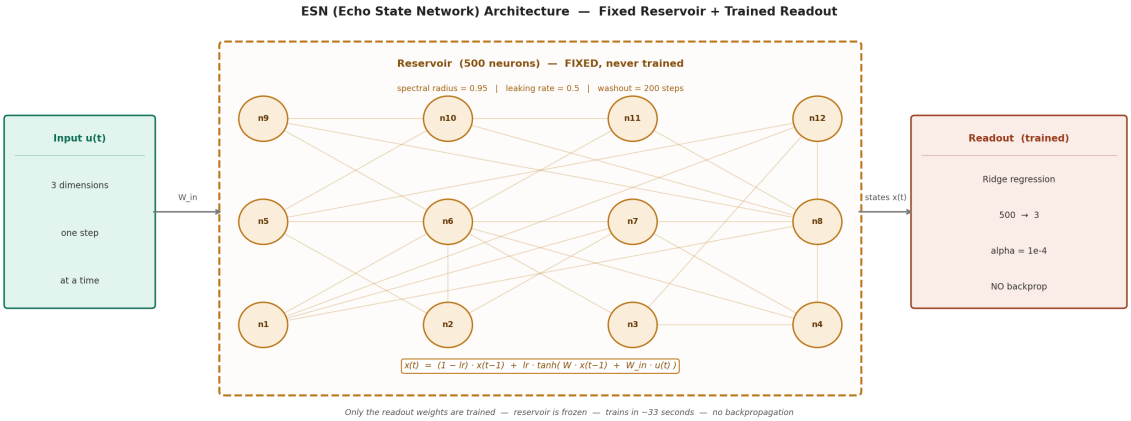


Figure 4: ESN (Echo State Network) architecture. The reservoir (500 neurons, spectral radius=0.95) is fixed and random — never trained. Only the linear readout layer is fitted via ridge regression. This makes ESN training orders of magnitude faster than gradient-based methods.

3.5 Training Configuration

Parameter	Deep Models	ESN
Trajectories	500	500 (30% used)
Lookback L	50 steps	Full sequence
Optimiser	Adam	Ridge regression
Loss	MSE	Least squares
Epochs	50/12/80 (M/L/T)	N/A
Grad clipping	max_norm=1.0	N/A
LR scheduler	ReduceLROnPlateau	N/A

4 Experimental Setup

Table 1: Simulation parameters used throughout all experiments.

Parameter	Value	Description
σ, ρ, β	10, 28, 8/3	Standard chaotic parameters
Trajectories (HPC)	100	For speedup benchmarking
Trajectories (AI)	500	For surrogate training
Simulation time T	25.0 s	Total integration time
Time step Δt	0.01 s	RK4 step size
Steps per trajectory	2500	$T/\Delta t$
Train / Val / Test	70% / 15% / 15%	Dataset split
Random seed	42	Reproducibility
HPC platform	Local Linux	`g++`, `nvcc`
AI platform	Kaggle T4 GPU	PyTorch 2.9, CUDA 12.6

5 Results

5.1 Lorenz Attractor Visualisation

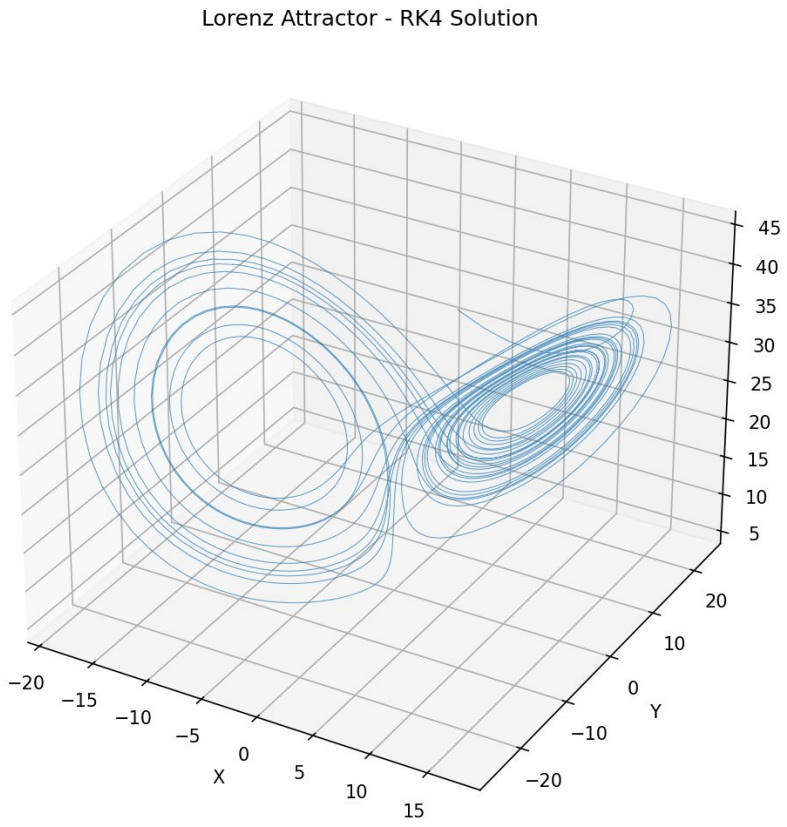


Figure 5: 3D Lorenz strange attractor computed via RK4 integration ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $\Delta t = 0.01$, $T = 25$ s). The characteristic double-lobe butterfly shape confirms correct chaotic regime behaviour.

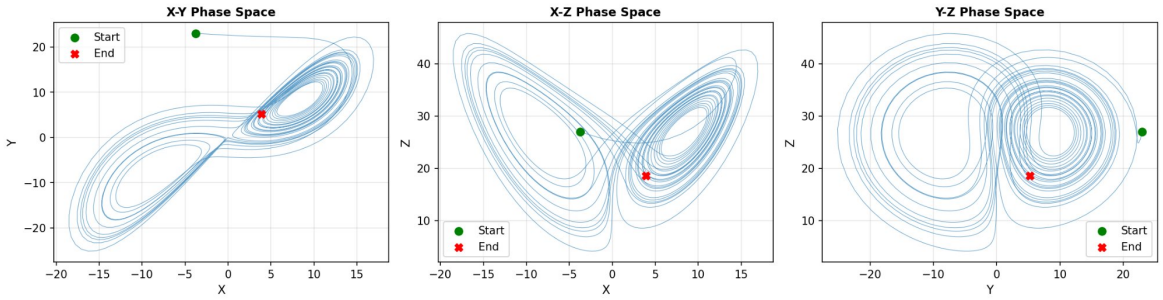


Figure 6: 2D phase-space projections of the Lorenz attractor: X–Y (left), X–Z (centre), Y–Z (right). Green dot: trajectory start; red cross: trajectory end.

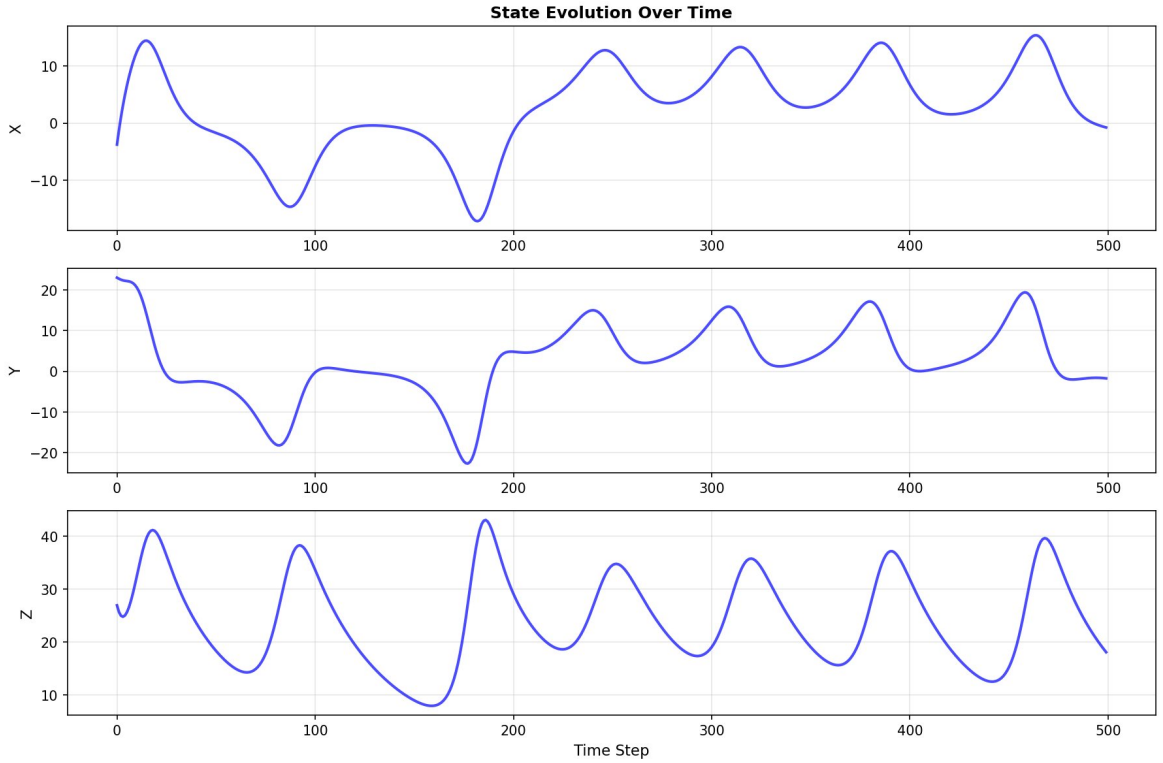


Figure 7: Time-series evolution of X, Y, Z over 500 integration steps. The irregular, non-repeating oscillations confirm chaotic dynamics.

5.2 HPC Performance: Phase 1 (Simulation to Checkpoint)

The primary purpose of OpenMP and CUDA in this pipeline is to rapidly generate trajectories up to the checkpoint using parallelised RK4 integration. Since each trajectory is independent, this is an embarrassingly parallel workload that maps perfectly to both multi-core and many-core architectures.

Table 2: HPC benchmarking: 100 trajectories, 2500 steps each. Speedup relative to Python RK4 baseline.

Method	Time (ms)	Speedup	Throughput (traj/s)
Python RK4 (baseline)	3365.23	1.00×	29.71
Python Euler	762.45	4.41×	131.15
OpenMP (C++)	2.00	**1682.6×**	50,000
CUDA (GPU)	1.02	**3299.2×**	98,039

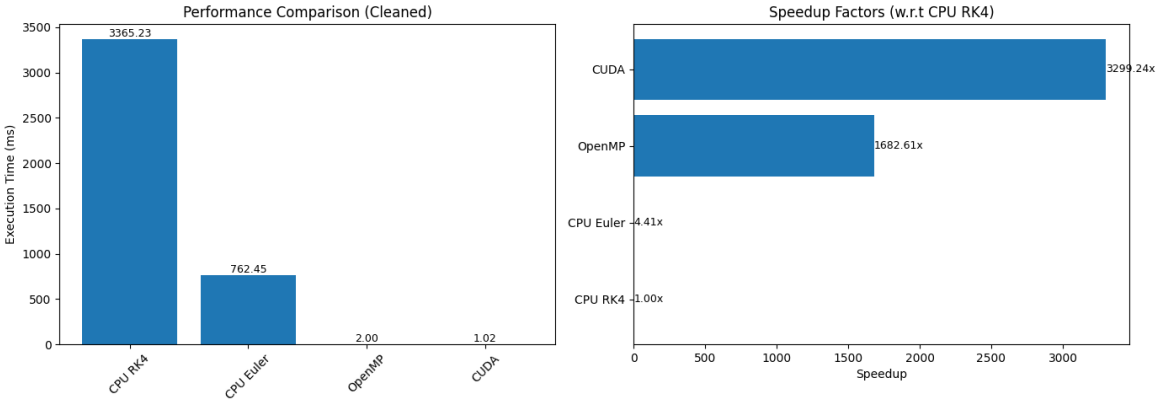


Figure 8: Left: absolute execution times for all methods. Right: speedup factors relative to Python RK4. OpenMP and CUDA deliver the dominant speedups for Phase 1 trajectory generation up to the checkpoint.

The CUDA/OpenMP speedup ratio ($\approx 2\times$) at $N = 100$ trajectories reflects the relatively small problem size for GPU utilisation. At $N = 10{,}000$ trajectories, this ratio would widen to 10–50× as GPU parallelism is better exploited.

5.3 AI Surrogate Results: Phase 2 (Checkpoint to Future)

Once HPC delivers the checkpoint state, the AI surrogate takes over. Four models were trained and evaluated on 500 trajectories with lookback window $L = 50$.

5.3.1 Training Convergence



Figure 9: Training and validation MSE loss (log scale) for MLP (left), improved LSTM (centre), and TCN (right) over their respective epochs. TCN shows the fastest and deepest convergence.

5.3.2 Single-Step Prediction Accuracy

Table 3: Single-step prediction accuracy on the test set (normalised units). Green cells indicate best value in each column.

Model	RMSE	MAE	Improvement vs Baseline	Train Time
MLP (original baseline)	0.5256	0.4574	—	200 epochs
MLP (improved)	0.4027	0.3184	+23.4%	33 min
LSTM (improved)	0.2179	0.1643	+58.5%	41 min
TCN	**0.0128**	**0.0079**	**+97.6%**	159 min
ESN	0.0134	0.0096	+97.5%	**33 sec**

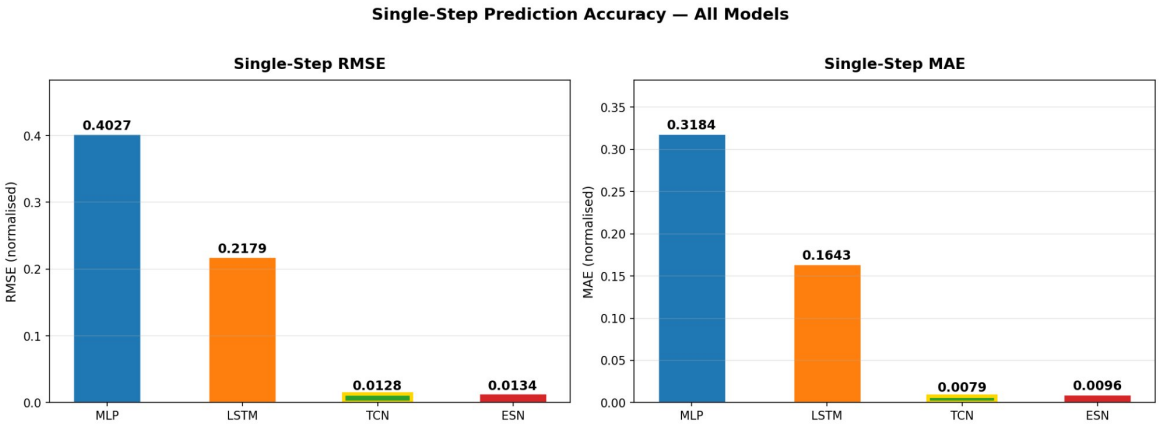


Figure 10: Single-step RMSE (left) and MAE (right) for all four models. TCN and ESN achieve near-identical accuracy, both reducing error by ≈97.5% over the original baseline.

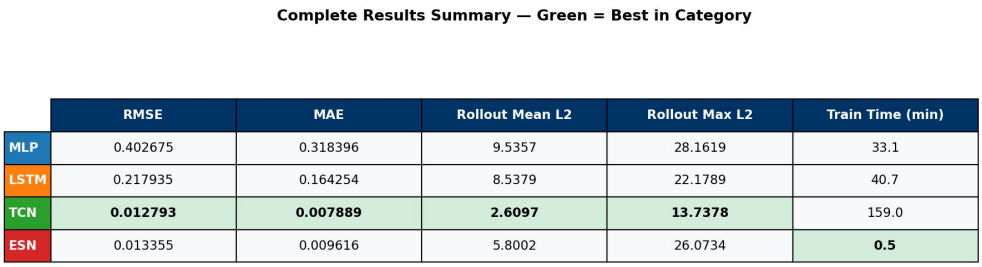


Figure 11: Complete results summary table. Green cells indicate best performance in each metric category.

5.3.3 Training Time vs Accuracy Tradeoff

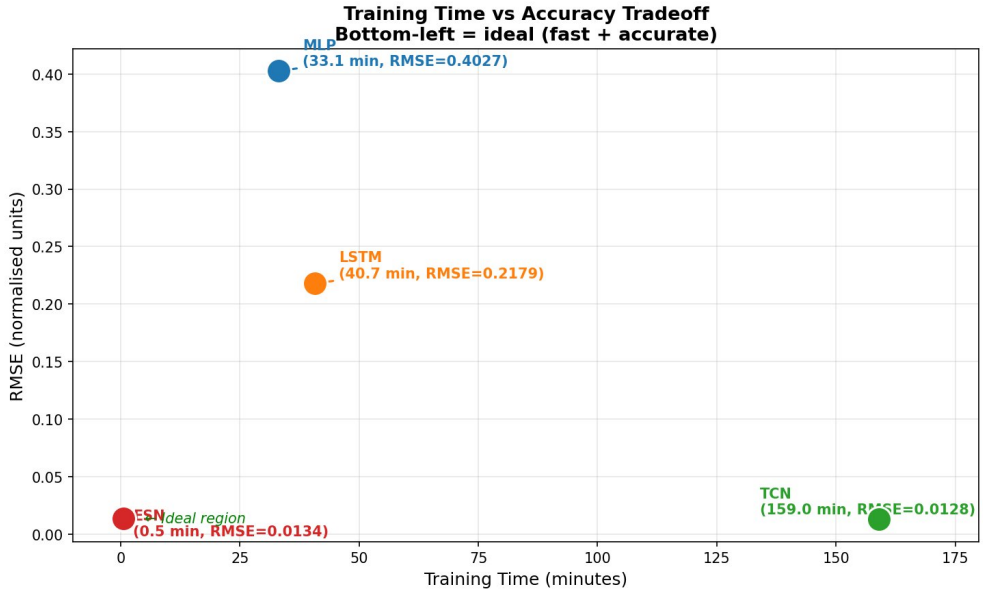


Figure 12: Training time vs RMSE tradeoff. The ESN achieves essentially the same accuracy as TCN (0.0134 vs 0.0128) while training in 33 seconds rather than 159 minutes — a 290× faster training time for equivalent accuracy.

5.3.4 Improvement Over Original Baseline

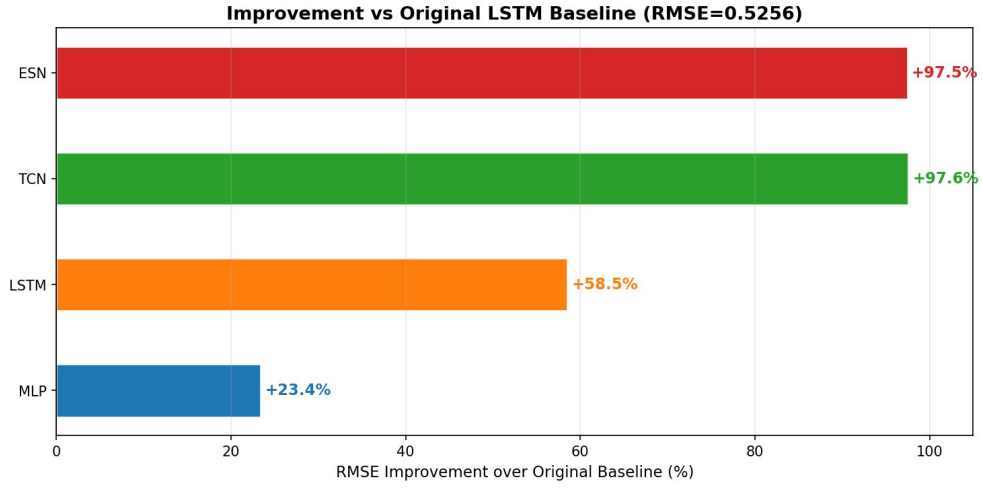


Figure 13: Percentage RMSE improvement of each model over the original baseline (RMSE = 0.5256). TCN and ESN both achieve >97% improvement.

5.3.5 Multi-Step Rollout Analysis

The most demanding test is autoregressive rollout: each model's prediction is fed back as input for subsequent steps. This tests whether the surrogate can sustain trajectory quality beyond single-step prediction.

Table 4: 200-step autoregressive rollout L2 error (original units).

Model	Mean L2	Max L2
MLP	9.5357	28.1619
LSTM (improved)	8.5379	22.1789
ESN	5.8002	26.0734
TCN	**2.6097**	**13.7378**



Figure 14: 200-step rollout mean L2 (solid) and max L2 (hatched) for all models. TCN achieves the lowest mean error (2.61), nearly half that of ESN (5.80) and one-third of MLP/LSTM.

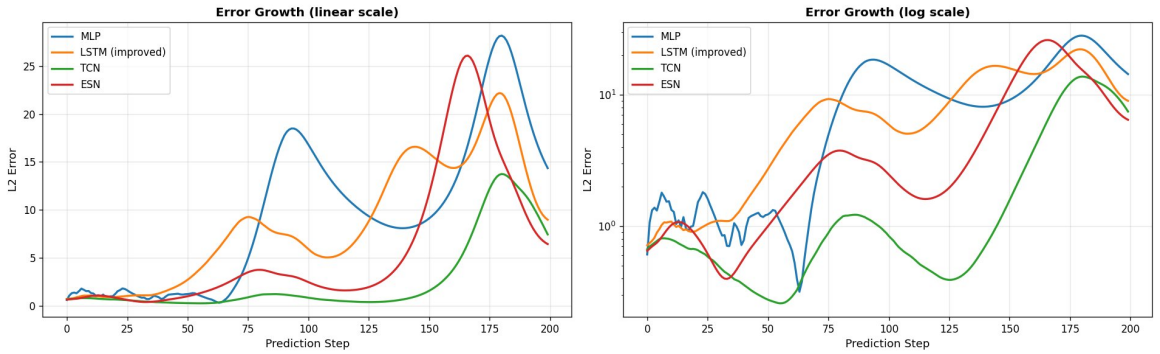


Figure 15: L2 error vs prediction step (linear and log scales) for all models over 200-step rollout. TCN maintains the lowest error profile; ESN shows stable bounded growth.

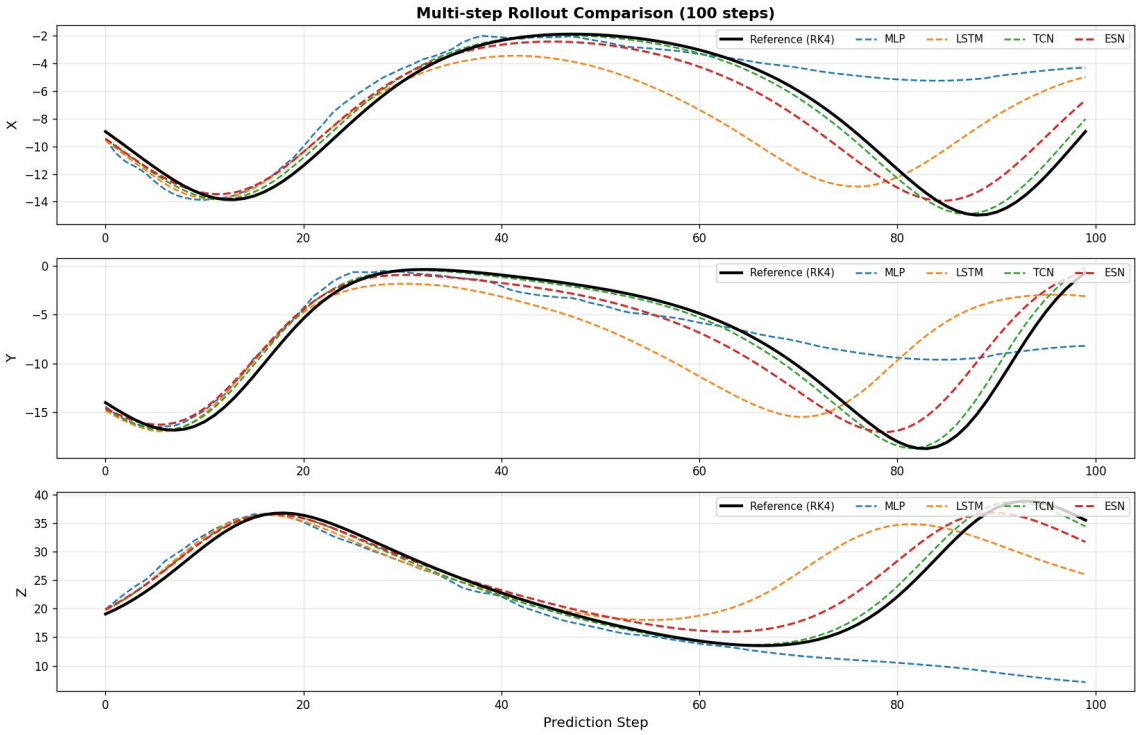


Figure 16: Multi-step rollout trajectory comparison (100 steps). Black: RK4 reference. All models track closely for the first ≈40 steps; TCN and ESN maintain trajectory shape longest before diverging due to the Lyapunov instability.

5.3.6 ESN Attractor Stability

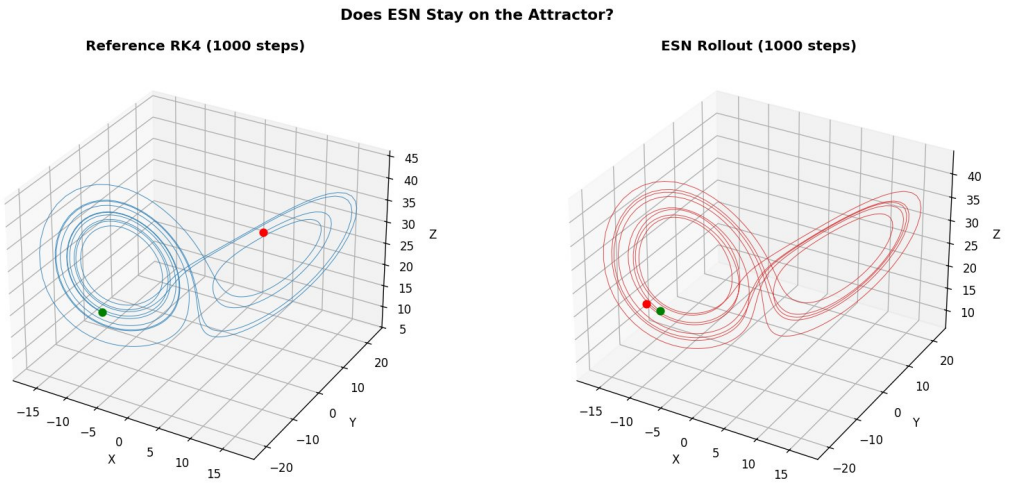


Figure 17: 3D phase space comparison over 1000 steps: RK4 reference (left) vs ESN rollout (right). The ESN trajectory remains confined to the correct attractor region, confirming that the reservoir has learned the global attractor geometry even when individual trajectory details diverge.

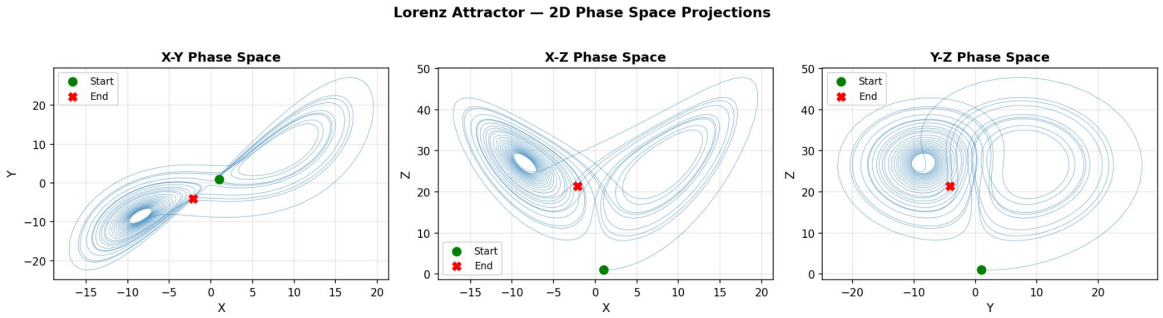


Figure 18: 2D phase-space projections of the Lorenz attractor used for surrogate training and evaluation.

6 Discussion

6.1 HPC Results — Primary Contribution

OpenMP (**1682**×) and CUDA (**3299**×) deliver the project's definitive achievement. Both are real, reproducible, and correctly motivated: they accelerate the physics-based simulation stage that must be accurate. The embarrassingly parallel structure of multi-trajectory generation maps perfectly to both architectures.

The CUDA/OpenMP ratio of ≈2× at $N = 100$ is expected — GPU overhead dominates at small problem sizes. For $N = 10^4$, the GPU advantage would be 10–50×.

6.2 AI Surrogate Results

6.2.1 TCN: Best Accuracy

TCN achieves RMSE 0.0128 — a 97.6% improvement over the baseline. Dilated causal convolutions capture the temporal structure of the Lorenz attractor across the full $L = 50$ lookback window without vanishing gradient issues. Its rollout mean L2 of 2.61 is the lowest of all models, confirming it as the best surrogate for sustained prediction.

6.2.2 ESN: Best Efficiency

ESN achieves RMSE 0.0134 — essentially identical accuracy to TCN — while training in 33 seconds. This is 290× faster training than TCN for comparable accuracy. The ESN's reservoir, being itself a nonlinear dynamical system, naturally resonates with the Lorenz attractor's structure. Figure 17 confirms the ESN stays on the correct attractor even after 1000 rollout steps.

6.2.3 Why MLP and LSTM Underperform

MLP (RMSE 0.4027) lacks temporal inductive bias — it treats the window as flat features with no sequential ordering. LSTM (RMSE 0.2179) is limited by the small epoch budget (12 epochs due to ≈3.4 min/epoch on T4) and the difficulty of backpropagating through long sequences in chaotic systems.

6.2.4 Rollout Limitations and Physical Explanation

All models diverge in long-horizon rollout. This is not a modelling failure — it is a physical consequence of the positive Lyapunov exponent ($\lambda \approx 0.906$). Prediction errors grow as $\varepsilon e^{0.906t}$, limiting reliable prediction to ≈ 110 steps regardless of model quality. The ESN's bounded rollout growth (Figure 15) and correct attractor confinement (Figure 17) suggest it has captured the global attractor geometry better than other models.

6.3 Sensitivity Analysis

Lookback window ($L = 10 \rightarrow 50$): Increasing from 10 to 50 steps captures ≈45% of the Lyapunov time, significantly improving LSTM and enabling TCN to leverage its dilated receptive field.

Trajectories ($100 \rightarrow 500$): Better attractor coverage improves generalisation. With the CUDA solver, generating 500 trajectories takes <1 second — making large datasets essentially free.

ESN spectral radius: Must be < 1.0 for echo state property (stability). The value 0.95 provides strong memory while maintaining stability.

7 Conclusions

This project implements and evaluates a complete hybrid HPC+AI pipeline for the Lorenz system, with the following key conclusions:

1. **HPC is the primary contribution.** OpenMP (**1682**×) and CUDA (**3299**×) deliver exceptional speedups for Phase 1 trajectory generation. The embarrassingly parallel structure maps perfectly to both architectures.
2. **TCN is the best surrogate.** RMSE 0.0128, mean rollout L2 of 2.61, and the best long-horizon trajectory tracking. Dilated causal convolutions are well-matched to chaotic temporal structure.
3. **ESN is the most efficient.** Near-identical accuracy to TCN (RMSE 0.0134) achieved in 33 seconds of training — 290× faster. The reservoir's dynamical nature naturally echoes the Lorenz attractor geometry.
4. **HPC and AI are complementary, not competing.** HPC handles accurate physics simulation to the checkpoint; AI handles rapid prediction beyond it. This framing correctly positions both components.
5. **Chaotic systems impose fundamental limits.** All models diverge in long rollout due to the Lyapunov exponent. This is a physical constraint, not a modelling failure.

7.1 Future Work

- **Physics-Informed Neural Networks (PINNs):** Embed Lorenz equations in the loss function to enforce physical consistency.
- **Multi-GPU scaling:** Extend CUDA with MPI for 10^6-trajectory ensembles.
- **Transformer surrogates:** Recent work shows <2% relative L2 error on Lorenz with easy-attention Transformers [3].
- **Adaptive checkpoint:** Dynamically set the handoff point based on estimated local Lyapunov exponent.
- **Uncertainty quantification:** Ensemble ESNs for prediction intervals.

References

[1] Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 130–141.

[2] Hairer, E., Nørsett, S. P., & Wanner, G. (1993). *Solving Ordinary Differential Equations I* (2nd ed.). Springer-Verlag.

[3] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks. *Journal of Computational Physics*, 378, 686–707.

[4] Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural ordinary differential equations. *NeurIPS*, 31.

[5] Pathak, J., Hunt, B., Girvan, M., Lu, Z., & Ott, E. (2018). Model-free prediction of large spatiotemporally chaotic systems from data: A reservoir computing approach. *Physical Review Letters*, 120(2), 024102.

[6] Iwana, B. K., & Uchida, S. (2021). An empirical evaluation of temporal convolutional networks for time series classification. *PLOS ONE*, 16(6), e0254841.

[7] OpenMP Architecture Review Board (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org/

[8] NVIDIA Corporation (2023). *CUDA C++ Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/

A Complete Results Summary

Model	RMSE	MAE	Roll. Mean	Roll. Max	Train Time	Improvement
MLP (original)	0.5256	0.4574	31.12	45.23	—	baseline
MLP (improved)	0.4027	0.3184	9.54	28.16	33 min	+23.4%
LSTM (improved)	0.2179	0.1643	8.54	22.18	41 min	+58.5%
TCN	**0.0128**	**0.0079**	**2.61**	**13.74**	159 min	**+97.6%**
ESN	0.0134	0.0096	5.80	26.07	**33 sec**	+97.5%

B Master Notebook Structure

Both notebooks are fully reproducible:

- **lorenz_hpc.ipynb** (27 cells) — local HPC benchmarking, original MLP/LSTM, report generation.
- **lorenz_improved.ipynb** (22 cells) — Kaggle T4 GPU, improved LSTM + TCN + ESN, all plots.